

How COMMAND Analytics works

Under the hood, with the code that does it

docs/use-case-2/21-under-the-hood-brief.md · rendered for review

You asked how this works under the hood. Every code block below is quoted exactly from a file running on your instance, with the file and line number, so you can open it and check. Lines beginning `//` or `#` inside a block are the code's own comments; anything we have added to explain a block is outside it, in the text.

The shape of it. Nine server scripts, three browser scripts, one stylesheet and two pages, all installed inside ServiceNow as ordinary configuration records. No server of ours, no database of ours. You open a page, ServiceNow runs our code against your own tables, and sends back one finished page with the numbers already in it. **Nothing is exported.** It is one round trip, because we measured that the alternative does not work here: a request from a logged-in browser session to a Scripted REST endpoint never returns on this instance, and the platform's own Table API behaves the same way.

1. Every number is counted as *you*, not as an administrator

ServiceNow's fast counter, `GlideAggregate`, **does not apply row-level permissions** — it counts rows you are not allowed to open. That is the gap this product exists to close, so every page runs both counts and compares them.

```
var fast = this.fastCount(table, query); // CmdData.js:402
var proof = this.secureCountBoxed(table, query, CmdData.PROOF_MS); //:469
```

The first line is the indexed aggregate — fast, and blind to row permissions. The second walks the rows through `GlideRecordSecure`, which is the platform's own permission check, inside a time budget.

The difference between the two *is* the rows you cannot open. The page shows the verdict as a badge, top right: **ACL VERIFIED** (both agree), **FILTERED** (you are seeing only your rows), **BOUNDED** (the check ran out of time, so the number is a floor rather than a total), or **NO ACCESS**.

Measured on your development instance on **9 September 2026**, as a real user holding no roles, with no change made to the instance's security setup:

TABLE	WHAT NATIVE REPORTING COUNTS	WHAT THIS USER CAN ACTUALLY OPEN
incident	4,284	818
task	8,503	818
problem	544	0
change_request	1,505	0
sys_user	667	667 (the control)

Read the first row again: **native reporting shows that user 4,284 incidents when they can open 818** — an overstatement of 3,466 records. On `problem` our page shows a red **NO ACCESS** badge over *"This subject holds 544 records and your permissions do not admit any of them, so there is nothing to show."*

2. The chart is chosen by measuring the data, not from a stored list

Most tools save a chart type with the report. We measure the column first — how many distinct values, how full, how concentrated — then pick a form that shape can carry honestly.

The shape of one column, measured over the rows *you* are allowed to read — how many values it holds, how dominant the biggest one is, how populated it is:

```
var distinct = rows.length; // CmdData.js:1232
var topShare = (total > 0 && distinct > 0) ? rows[0].count / total : 0;
fill: total > 0 ? (total - emptyCount) / total : 1, // :1288
```

And the rules record *why* a form was refused, not only which one won:

```

if (c.zeroVariance && form !== 'stat_tile') {                                // CmdForm.js:344
    form = 'stat_tile';
    demoted = true;
    reason = 'every category holds the same value, so the distribution carries no information';
}

```

So one subject draws different charts as its data changes: six categories become a share-over-time chart, eleven become a grid of small charts, and a column holding one value becomes a single number with a sentence explaining itself.

3. Drilldown is offered only where the data supports it

ServiceNow's dictionary declares which fields are parent and child, but a declaration is not a fact about the rows: on one tenant `incident.subcategory` was declared under `category` and filled in on **42 of 13,986** records — a drill that leads nowhere 99.7% of the time. So every level is measured against *your* permitted rows first, and a refused level says why instead of being a dead click.

```

// CmdDrill.js:159 – a bounded scan proves presence and never absence
if (!prof.measured) {
    res.reason = 'not measured: the permission-checked scan ran out of ' +
        'time on this subject before it read any rows, so ' +
        'whether this is a level is unknown rather than no';
    return res;
}
if (prof.distinctNonEmpty < G.MIN_DISTINCT) {                                // CmdDrill.js:179
    res.reason = prof.capped
        ? 'the ' + prof.total + ' records read before the scan stopped all '

```

Only a complete scan is allowed to say *every*; a partial one says how far it got.

That first branch was added on **9 September 2026**, after an audit found the page claiming *"every record here has the same active"* about a column that actually holds 7,380 true and 1,123 false. The permission check had run out of time before reading a single row, and the page turned *"we did not look"* into *"there is nothing there."* A measurement of nothing must never become a claim about everything — that rule is now in the code and in the test suite.

4. How it reaches your instance, and how we know it landed

A ServiceNow write answers **HTTP 200 whether or not it stored what you sent**, so we never trust the status code: every write is read back and compared byte for byte, and every asset is fetched from the URL the page will request. Even that is not enough — a compile error leaves the record byte-perfect and the class undefined — so the deploy ends by asking the instance to construct each class:

```

js.append(                                                                # deploy.py:169
    "try { new %s(); out['%s'] = 'ok'; } "
    "catch (e) { out['%s'] = String(e).substring(0, 160); }" % (n, n, n)
)

```

What it does not do — said plainly, so nothing surprises you later

- **Not a Power BI replacement.** Power-BI-grade visual and interaction quality on ServiceNow-native data — but no joining across outside systems, and no free-form self-service modelling.
- **AI is optional and off by default.** Nothing on these pages is generated by a language model today. Switched on, it sees column statistics and the totals already on screen — never a record.
- **On large tables it is honest rather than fast.** Where row-by-row permission checking would take too long the page stops and shows **BOUNDED** — never a quiet count you are not entitled to.